

Software Engineer

Interview Preparation Notes

DSA · Full Stack · System Design · Study Plans

Comprehensive Notes for Final-Year CS Students & SWE Candidates

23 Chapters · 2 Study Plans · Interview Ready

TABLE OF CONTENTS

SECTION 1: DATA STRUCTURES & ALGORITHMS

- Ch 1 · Time and Space Complexity
- Ch 2 · Arrays and Strings
- Ch 3 · Searching and Sorting
- Ch 4 · Linked Lists
- Ch 5 · Stack and Queue
- Ch 6 · Trees and BST
- Ch 7 · Heaps and Priority Queues
- Ch 8 · Graphs
- Ch 9 · Hashing
- Ch 10 · Recursion and Backtracking
- Ch 11 · Dynamic Programming
- Ch 12 · Greedy Algorithms
- Ch 13 · Fix-the-Code Round

SECTION 2: FULL STACK DEVELOPMENT

- Ch 14 · JavaScript Basics
- Ch 15 · Node.js Basics
- Ch 16 · React.js – Basic to Advanced
- Ch 17 · Express.js
- Ch 18 · Spring Boot
- Ch 19 · Databases & Advanced DBMS
- Ch 20 · Redis
- Ch 21 · RabbitMQ
- Ch 22 · Docker
- Ch 23 · CI/CD Pipeline

SECTION 3: STUDY PLANS

- Plan A · 15-Day DSA Roadmap
- Plan B · 15-Day Java + Full Stack Roadmap

SECTION 1: DATA STRUCTURES & ALGORITHMS

Chapter 1: Time and Space Complexity

1.1 Why Complexity Analysis?

When you write code, two things matter for performance: **how fast** it runs (time complexity) and **how much memory** it uses (space complexity). Big-O notation lets us describe these in a machine-independent way.

1.2 Asymptotic Notations

Notation	Meaning	Description	Example
O (Big-O)	Upper bound	Worst-case guarantee	$O(n^2)$ for Bubble Sort
Ω (Omega)	Lower bound	Best-case guarantee	$\Omega(n)$ for linear search
Θ (Theta)	Tight bound	Both upper & lower	$\Theta(n \log n)$ for Merge Sort
o (little-o)	Strict upper	Grows strictly slower	$o(n^2)$ means $< n^2$
ω (little- ω)	Strict lower	Grows strictly faster	$\omega(n)$ means $> n$

1.3 Common Complexity Classes (Best → Worst)

Complexity	Name	Example Algorithm	$n=1000$ ops (approx)
$O(1)$	Constant	Array access, HashMap get	1
$O(\log n)$	Logarithmic	Binary Search	10
$O(n)$	Linear	Linear Search, one loop	1,000
$O(n \log n)$	Linearithmic	Merge Sort, Heap Sort	10,000
$O(n^2)$	Quadratic	Bubble Sort, nested loops	1,000,000
$O(n^3)$	Cubic	Matrix Multiplication (naive)	10 ⁹ (slow!)
$O(2^n)$	Exponential	Subsets, brute-force DP	Astronomical
$O(n!)$	Factorial	Permutations, TSP brute	Astronomical

1.4 Rules for Calculating Big-O

- **Drop constants:** $O(2n) \rightarrow O(n)$
- **Drop lower-order terms:** $O(n^2 + n) \rightarrow O(n^2)$
- **Sequential steps ADD:** $O(n) + O(m) \rightarrow O(n + m)$
- **Nested loops MULTIPLY:** $O(n)$ inside $O(n) \rightarrow O(n^2)$
- **Recursion:** Use recurrence relations (Master Theorem)

1.5 Master Theorem for Divide & Conquer

For $T(n) = aT(n/b) + f(n)$:

Case	Condition	Result	Example
Case 1	$f(n) = O(n^{\log_b a - \epsilon})$	$T(n) = \Theta(n^{\log_b a})$	$T(n) = 2T(n/2) + 1 \rightarrow O(n)$
Case 2	$f(n) = \Theta(n^{\log_b a})$	$T(n) = \Theta(n^{\log_b a} \cdot \log n)$	$T(n) = 2T(n/2) + n \rightarrow O(n \log n)$
Case 3	$f(n) = \Omega(n^{\log_b a + \epsilon})$	$T(n) = \Theta(f(n))$	$T(n) = T(n/2) + n \rightarrow O(n)$

1.6 Space Complexity

- **Auxiliary space:** extra memory used by algorithm (not input)
- Recursive calls use $O(\text{depth})$ stack space
- In-place algorithms use $O(1)$ auxiliary space
- Memoization tables use $O(n)$ or $O(n^2)$ extra space

$O(n)$ space – storing result array

```
def double_array(arr):
    return [x * 2 for x in arr]    # new list of size n
```

$O(1)$ space – in-place

```
def double_inplace(arr):
    for i in range(len(arr)):
        arr[i] *= 2
```

1.7 Practice Problems

- What is the time complexity of two nested loops running $n/2$ times each? $\rightarrow O(n^2)$
- Binary search on sorted array of size n ? $\rightarrow O(\log n)$
- Finding all subsets of a set of n elements? $\rightarrow O(2^n)$
- Counting digits in a number N ? $\rightarrow O(\log N)$

■ **TIP:** Always state the Big-O for every function you write in an interview.

■ **COMMON MISTAKE:** Forgetting to account for recursive call stack space is a very common mistake.

■ Most Asked Interview Questions

Q: What is the difference between $O(n)$ and $\Theta(n)$?

A: $O(n)$ is an upper bound (worst case); $\Theta(n)$ is a tight bound meaning the algorithm is BOTH $O(n)$ and $\Omega(n)$.

Q: What is the time complexity of accessing an element in a HashMap?

A: $O(1)$ average case, $O(n)$ worst case due to hash collisions.

Q: How do you analyze a recursive algorithm?

A: Write the recurrence relation and apply the Master Theorem or substitution/recursion tree method.

Q: What is amortized analysis?

A: Average time per operation over a sequence of operations. E.g., dynamic array append is $O(1)$ amortized even though occasional resize is $O(n)$.

■ QUICK REVISION

- O = upper bound (worst), Ω = lower bound (best), Θ = tight bound
- Common: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$
- Drop constants and lower-order terms
- Nested loops multiply; sequential steps add

- Recursion adds $O(\text{depth})$ stack space

Chapter 2: Arrays and Strings

2.1 Array Basics

An array is a contiguous block of memory storing elements of the same type. Accessing any element by index is $O(1)$. Insertion/deletion in the middle is $O(n)$.

Operation	Array	Dynamic Array (list)
Access by index	$O(1)$	$O(1)$
Search (unsorted)	$O(n)$	$O(n)$
Insert at end	$O(1)$	$O(1)$ amortized
Insert at middle	$O(n)$	$O(n)$
Delete at middle	$O(n)$	$O(n)$
Space	$O(n)$	$O(n)$

2.2 Prefix Sum

Build a prefix array so range-sum queries are $O(1)$ after $O(n)$ preprocessing.

```
# Prefix Sum
def build_prefix(arr):
    prefix = [0] * (len(arr) + 1)
    for i, v in enumerate(arr):
        prefix[i+1] = prefix[i] + v
    return prefix

def range_sum(prefix, l, r): # inclusive [l, r]
    return prefix[r+1] - prefix[l]

# Example
arr = [3, 1, 4, 1, 5, 9, 2, 6]
prefix = build_prefix(arr)
print(range_sum(prefix, 2, 5)) # sum of arr[2..5] = 4+1+5+9 = 19
```

2.3 Sliding Window

Used for problems on contiguous sub-arrays/sub-strings. Fixed window: maintain sum/count as window moves. Variable window: expand right, shrink left based on condition.

```
# Fixed window - max sum of k elements
def max_sum_k(arr, k):
    win = sum(arr[:k])
    best = win
    for i in range(k, len(arr)):
        win += arr[i] - arr[i-k]
        best = max(best, win)
    return best

# Variable window - longest sub-array with sum <= target
def longest_subarray(arr, target):
    l = total = best = 0
    for r in range(len(arr)):
        total += arr[r]
```

```

        while total > target:
            total -= arr[l]; l += 1
        best = max(best, r - l + 1)
    return best

```

2.4 Two Pointers

Two pointers iterate toward each other (sorted array) or together (same direction).

```

# Two-sum in sorted array
def two_sum_sorted(arr, target):
    l, r = 0, len(arr) - 1
    while l < r:
        s = arr[l] + arr[r]
        if s == target: return (l, r)
        elif s < target: l += 1
        else: r -= 1
    return None

```

```

# Remove duplicates in-place
def remove_dups(arr):
    if not arr: return 0
    w = 1
    for r in range(1, len(arr)):
        if arr[r] != arr[r-1]:
            arr[w] = arr[r]; w += 1
    return w

```

2.5 Kadane's Algorithm – Maximum Subarray Sum

```

def max_subarray(arr):
    best = cur = arr[0]
    for x in arr[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best    # Time O(n), Space O(1)

# Variant: also return the subarray indices
def max_subarray_idx(arr):
    best = cur = arr[0]
    start = end = s = 0
    for i in range(1, len(arr)):
        if arr[i] > cur + arr[i]:
            cur = arr[i]; s = i
        else:
            cur += arr[i]
        if cur > best:
            best = cur; start = s; end = i
    return best, start, end

```

2.6 String Manipulation Essentials

- Reverse: `s[::-1]` or two-pointer swap
- Check palindrome: compare `s` with reverse, or two-pointer
- Anagram check: sort both strings or compare frequency maps
- Substring search: naive $O(nm)$, KMP $O(n+m)$

```

# Anagram check
from collections import Counter

```

```
def is_anagram(s, t):
    return Counter(s) == Counter(t)    # O(n)

# Longest palindromic substring (expand-around-center)
def longest_palindrome(s):
    best = ""
    for i in range(len(s)):
        for l, r in [(i, i), (i, i+1)]:    # odd / even
            while l >= 0 and r < len(s) and s[l] == s[r]:
                if r - l + 1 > len(best): best = s[l:r+1]
                l -= 1; r += 1
    return best
```

■ Most Asked Interview Questions

Q: Find the maximum product subarray.

A: Track max AND min at each step (negative * negative = positive). O(n) time, O(1) space.

Q: Given an array, find all pairs with sum = k.

A: Use a HashSet. For each element x, check if k-x is in the set. O(n) time.

Q: Rotate an array by k positions.

A: Reverse entire array, reverse first k, reverse rest. O(n) time, O(1) space.

Q: Find the missing number in [0..n].

A: Expected sum = $n*(n+1)/2$. Subtract actual sum. O(n) time, O(1) space.

Q: Longest substring without repeating characters.

A: Sliding window + HashMap storing last index. O(n) time.

■ QUICK REVISION

- Prefix Sum → O(1) range queries after O(n) build
- Sliding Window → contiguous subarray problems
- Two Pointers → sorted arrays, sum problems, palindromes
- Kadane's → maximum subarray in O(n)
- String anagram → frequency map or sort comparison

Chapter 3: Searching and Sorting

3.1 Binary Search

Binary search works on **sorted** arrays. Halves the search space each step → $O(\log n)$.

```
# Iterative Binary Search
def binary_search(arr, target):
    l, r = 0, len(arr) - 1
    while l <= r:
        mid = l + (r - l) // 2    # avoids overflow
        if arr[mid] == target: return mid
        elif arr[mid] < target: l = mid + 1
        else: r = mid - 1
    return -1

# Find first occurrence
def first_occurrence(arr, target):
    l, r, res = 0, len(arr)-1, -1
    while l <= r:
        mid = (l + r) // 2
        if arr[mid] == target: res = mid; r = mid - 1
        elif arr[mid] < target: l = mid + 1
        else: r = mid - 1
    return res
```

3.2 Sorting Algorithms – Comparison Table

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Tim Sort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Yes

3.3 Merge Sort – Implementation

```
def merge_sort(arr):
    if len(arr) <= 1: return arr
    mid = len(arr) // 2
    L = merge_sort(arr[:mid])
    R = merge_sort(arr[mid:])
    return merge(L, R)

def merge(L, R):
    result = []; i = j = 0
    while i < len(L) and j < len(R):
        if L[i] <= R[j]: result.append(L[i]); i += 1
    result.extend(L[i:] + R[j:])
```

```

        else:
            result.append(R[j]); j += 1
    return result + L[i:] + R[j:]

```

3.4 Quick Sort – Implementation

```

def quick_sort(arr, lo=0, hi=None):
    if hi is None: hi = len(arr) - 1
    if lo < hi:
        p = partition(arr, lo, hi)
        quick_sort(arr, lo, p - 1)
        quick_sort(arr, p + 1, hi)

def partition(arr, lo, hi):
    pivot = arr[hi]
    i = lo - 1
    for j in range(lo, hi):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i+1], arr[hi] = arr[hi], arr[i+1]
    return i + 1

```

■ **TIP:** Use Merge Sort when stability matters. Use Quick Sort for in-place and average-case speed.

■ **COMMON MISTAKE:** Quick Sort degrades to $O(n^2)$ on already-sorted arrays without randomized pivot.

■ Most Asked Interview Questions

Q: When would you use Insertion Sort over Merge Sort?

A: For small arrays ($n < 20$) or nearly-sorted data. Insertion Sort has excellent cache performance and low overhead.

Q: How does Binary Search work on a rotated sorted array?

A: Find which half is sorted using mid comparison, then decide which half the target is in. $O(\log n)$.

Q: What is a stable sort and why does it matter?

A: Preserves relative order of equal elements. Important when sorting objects by multiple keys (e.g., sort by name then by age).

Q: How do you find duplicates in an array in $O(n)$ time and $O(1)$ space?

A: Floyd's cycle detection (if values in $[1, n]$), or XOR trick, or marking visited by negating value at index.

■ QUICK REVISION

- Binary Search: sorted array, $O(\log n)$, use $l + (r-l)//2$ to avoid overflow
- Merge Sort: $O(n \log n)$ always, stable, $O(n)$ space
- Quick Sort: $O(n \log n)$ average, in-place, worst $O(n^2)$ with bad pivot
- For nearly sorted data: Insertion Sort wins
- Counting Sort: $O(n+k)$ but only for integers in small range

Chapter 4: Linked Lists

4.1 Types of Linked Lists

Type	Description	Next Pointer	Prev Pointer	Last→Head
Singly	One direction	Yes	No	No
Doubly	Both directions	Yes	Yes	No
Circular	Tail points to head	Yes	No	Yes
Doubly Circular	Both + circular	Yes	Yes	Yes

4.2 Singly Linked List – Node & Basic Operations

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None

class LinkedList:
    def __init__(self): self.head = None

    def append(self, val):          # O(n)
        node = Node(val)
        if not self.head: self.head = node; return
        cur = self.head
        while cur.next: cur = cur.next
        cur.next = node

    def prepend(self, val):        # O(1)
        node = Node(val)
        node.next = self.head
        self.head = node

    def delete(self, val):         # O(n)
        if not self.head: return
        if self.head.val == val: self.head = self.head.next; return
        cur = self.head
        while cur.next and cur.next.val != val: cur = cur.next
        if cur.next: cur.next = cur.next.next
```

4.3 Reverse a Linked List (Iterative + Recursive)

```
# Iterative - O(n) time, O(1) space
def reverse(head):
    prev, cur = None, head
    while cur:
        nxt = cur.next
        cur.next = prev
        prev, cur = cur, nxt
    return prev

# Recursive
def reverse_rec(head):
    if not head or not head.next: return head
    new_head = reverse_rec(head.next)
```

```

head.next.next = head
head.next = None
return new_head

```

4.4 Detect Cycle – Floyd's Algorithm

```

def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast: return True
    return False

# Find cycle start
def cycle_start(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next; fast = fast.next.next
        if slow == fast:
            slow = head
            while slow != fast:
                slow = slow.next; fast = fast.next
            return slow # cycle start node
    return None

```

4.5 Merge Two Sorted Linked Lists

```

def merge_sorted(l1, l2):
    dummy = Node(0); cur = dummy
    while l1 and l2:
        if l1.val <= l2.val: cur.next = l1; l1 = l1.next
        else: cur.next = l2; l2 = l2.next
        cur = cur.next
    cur.next = l1 or l2
    return dummy.next

```

4.6 Find Middle Node (Fast/Slow Pointer)

```

def find_middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow # for even-length list, returns second middle

```

■ Most Asked Interview Questions

Q: How do you find the k-th node from the end?

A: Move one pointer k steps ahead, then move both until the front pointer reaches the end.

Q: How do you check if a linked list is a palindrome?

A: Find middle, reverse second half, compare both halves, restore list.

Q: How do you detect and remove a cycle?

A: Floyd's algorithm to detect; then reset one pointer to head and advance both by 1 to find entry point. Set next of the node before entry to None.

Q: Difference between array and linked list?

A: Array: random access $O(1)$, fixed size (or resizable). Linked List: sequential access $O(n)$, dynamic size, $O(1)$ insert/delete at head.

■ QUICK REVISION

- Fast/slow pointer → middle, cycle detection, k-th from end
- Reverse iteratively ($O(1)$ space) preferred in interviews
- Floyd's cycle: slow+1, fast+2; if meet → cycle exists
- Merge sorted lists with a dummy head node trick
- Always handle edge cases: empty list, single node

Chapter 5: Stack and Queue

5.1 Stack – LIFO

```
# Stack using Python list
class Stack:
    def __init__(self): self.data = []
    def push(self, x): self.data.append(x) # O(1)
    def pop(self): return self.data.pop() # O(1)
    def peek(self): return self.data[-1] # O(1)
    def is_empty(self): return len(self.data)==0

# Valid Parentheses (LeetCode 20)
def is_valid(s):
    stack = []; match = {'(': ')', '[': ']', '{': '}'}
    for c in s:
        if c in '([{': stack.append(c)
        elif not stack or stack[-1] != match[c]: return False
        else: stack.pop()
    return not stack
```

5.2 Queue – FIFO

```
from collections import deque

class Queue:
    def __init__(self): self.data = deque()
    def enqueue(self, x): self.data.append(x) # O(1)
    def dequeue(self): return self.data.popleft() # O(1)
    def peek(self): return self.data[0]
    def is_empty(self): return len(self.data)==0
```

5.3 Monotonic Stack – Next Greater Element

```
def next_greater(arr):
    n = len(arr); result = [-1] * n; stack = []
    for i in range(n):
        while stack and arr[stack[-1]] < arr[i]:
            result[stack.pop()] = arr[i]
        stack.append(i)
    return result
# arr = [4, 5, 2, 10] → result = [5, 10, 10, -1]
```

5.4 LRU Cache (HashMap + Doubly Linked List)

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.cache = OrderedDict()

    def get(self, key):
        if key not in self.cache: return -1
        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key, value):
```

```

if key in self.cache: self.cache.move_to_end(key)
self.cache[key] = value
if len(self.cache) > self.cap:
    self.cache.popitem(last=False)    # remove LRU

```

5.5 Circular Queue

```

class CircularQueue:
    def __init__(self, k):
        self.q = [0]*k; self.head=self.tail=-1; self.size=k

    def enqueue(self, val):
        if (self.tail+1)%self.size == self.head: return False    # full
        if self.head == -1: self.head = 0
        self.tail = (self.tail+1) % self.size
        self.q[self.tail] = val; return True

    def dequeue(self):
        if self.head == -1: return False    # empty
        if self.head == self.tail: self.head=self.tail=-1
        else: self.head = (self.head+1) % self.size
        return True

```

■ Most Asked Interview Questions

Q: Implement a stack using two queues.

A: Push to q1. For pop, move all but last element of q1 to q2, return last of q1, swap q1 and q2.

Q: Implement a queue using two stacks.

A: Enqueue to stack1. For dequeue, if stack2 empty, pop all stack1 into stack2, then pop stack2. Amortized O(1).

Q: Sliding window maximum using deque.

A: Maintain a monotonic decreasing deque of indices. Remove front if out of window. Remove back while smaller than current. Front is the max.

Q: What is a priority queue?

A: A queue where elements are dequeued based on priority (not arrival order). Implemented with a heap. O(log n) insert/delete, O(1) peek.

■ QUICK REVISION

- Stack: LIFO — push/pop/peek O(1)
- Queue: FIFO — enqueue/dequeue O(1) with deque
- Monotonic Stack → next greater/smaller element patterns
- LRU Cache → OrderedDict or HashMap + DLL, all ops O(1)
- Circular Queue → (tail+1)%size == head means FULL

Chapter 6: Trees and BST

6.1 Binary Tree Terminology

- Root: topmost node with no parent
- Leaf: node with no children
- Height: longest path from node to leaf
- Depth: distance from root to node
- Complete Binary Tree: all levels full except last (left-filled)
- Perfect Binary Tree: all levels completely filled
- Balanced Binary Tree: $|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$ for all nodes

6.2 Tree Traversals

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val; self.left = left; self.right = right

# Inorder: Left → Root → Right (gives sorted order in BST)
def inorder(root):
    if not root: return []
    return inorder(root.left) + [root.val] + inorder(root.right)

# Preorder: Root → Left → Right (used to copy/serialize tree)
def preorder(root):
    if not root: return []
    return [root.val] + preorder(root.left) + preorder(root.right)

# Postorder: Left → Right → Root (used to delete tree)
def postorder(root):
    if not root: return []
    return postorder(root.left) + postorder(root.right) + [root.val]

# Level-order BFS
from collections import deque
def level_order(root):
    if not root: return []
    q = deque([root]); result = []
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        result.append(level)
    return result
```

6.3 BST Operations

```
def bst_search(root, target):
    if not root or root.val == target: return root
    if target < root.val: return bst_search(root.left, target)
    return bst_search(root.right, target)
def bst_insert(root, val):
```

```

    if not root: return TreeNode(val)
    if val < root.val: root.left = bst_insert(root.left, val)
    else:
        root.right = bst_insert(root.right, val)
    return root

def bst_delete(root, key):
    if not root: return None
    if key < root.val: root.left = bst_delete(root.left, key)
    elif key > root.val: root.right = bst_delete(root.right, key)
    else:
        if not root.left: return root.right
        if not root.right: return root.left
        # Replace with inorder successor (min of right subtree)
        succ = root.right
        while succ.left: succ = succ.left
        root.val = succ.val
        root.right = bst_delete(root.right, succ.val)
    return root

```

6.4 Lowest Common Ancestor (LCA)

```

def lca(root, p, q):
    if not root or root == p or root == q: return root
    left = lca(root.left, p, q)
    right = lca(root.right, p, q)
    if left and right: return root # p and q on different sides
    return left or right

```

6.5 Diameter of a Binary Tree

```

def diameter(root):
    best = [0]
    def height(node):
        if not node: return 0
        lh = height(node.left)
        rh = height(node.right)
        best[0] = max(best[0], lh + rh)
        return 1 + max(lh, rh)
    height(root)
    return best[0]

```

■ Most Asked Interview Questions

Q: How do you check if a binary tree is balanced?

A: For each node, recursively get the height of left and right subtrees. If difference > 1, not balanced. Return -1 to propagate failure. O(n).

Q: Convert a sorted array to a height-balanced BST.

A: Recursively pick the middle element as root, then build left and right subtrees from subarrays.

Q: Serialize and deserialize a binary tree.

A: Use preorder traversal with null markers. BFS level-order is also acceptable.

Q: What is the difference between a complete and full binary tree?

A: Full: every node has 0 or 2 children. Complete: all levels filled except last, which is filled left to right.

■ QUICK REVISION

- Inorder of BST = sorted sequence
- LCA: if both p,q in same subtree → recurse; else current root is LCA

- Tree height: $1 + \max(\text{left_height}, \text{right_height})$
- BFS (level-order) uses a queue; DFS uses recursion or explicit stack
- BST search/insert/delete: $O(h)$ where $h = \log n$ balanced, n worst

Chapter 7: Heaps and Priority Queues

7.1 Heap Properties

- A heap is a **complete binary tree** stored as an array.
- Min-Heap: $\text{parent} \leq \text{children}$. Root is the minimum.
- Max-Heap: $\text{parent} \geq \text{children}$. Root is the maximum.
- For node at index i : left child = $2i+1$, right child = $2i+2$, parent = $(i-1)//2$

7.2 Heap Operations

Operation	Time Complexity	Description
insert (push)	$O(\log n)$	Add to end, bubble up
extract-min/max	$O(\log n)$	Remove root, replace with last, bubble down
peek	$O(1)$	Return root without removing
heapify	$O(n)$	Build heap from array (bottom-up)
heap sort	$O(n \log n)$	Sort using max-heap

```
import heapq

# Python uses min-heap by default
h = []
heapq.heappush(h, 3)
heapq.heappush(h, 1)
heapq.heappush(h, 5)
print(heapq.heappop(h))  # 1 (minimum)

# Max-heap: negate values
max_h = []
for v in [3, 1, 5]:
    heapq.heappush(max_h, -v)
print(-heapq.heappop(max_h))  # 5 (maximum)

# Build heap in  $O(n)$ 
arr = [5, 3, 8, 1, 9]
heapq.heapify(arr)  # in-place
```

7.3 Top K Elements Pattern

```
# K largest elements using min-heap of size k
def k_largest(nums, k):
    heap = nums[:k]
    heapq.heapify(heap)
    for n in nums[k:]:
        if n > heap[0]:
            heapq.heapreplace(heap, n)
    return heap  # Time  $O(n \log k)$ , Space  $O(k)$ 

# K-th largest element
def kth_largest(nums, k):
    return heapq.nlargest(k, nums)[-1]  #  $O(n \log k)$ 
```

7.4 Merge K Sorted Lists

```
def merge_k_sorted(lists):
    heap = []
    for i, lst in enumerate(lists):
        if lst:
            heapq.heappush(heap, (lst[0], i, 0))
    result = []
    while heap:
        val, i, j = heapq.heappop(heap)
        result.append(val)
        if j + 1 < len(lists[i]):
            heapq.heappush(heap, (lists[i][j+1], i, j+1))
    return result    # O(N log k) where N = total elements
```

■ Most Asked Interview Questions

Q: Find the median from a data stream.

A: Use two heaps: max-heap for lower half, min-heap for upper half. Balance their sizes. Median = average of tops or top of larger heap. $O(\log n)$ insert, $O(1)$ median.

Q: What is heapify and what is its time complexity?

A: Bottom-up heap construction. Start from last internal node and sift down. $O(n)$ amortized, NOT $O(n \log n)$.

Q: When to use a heap vs sorting?

A: Heap is better when you only need top-k elements ($O(n \log k)$) or a running min/max. Full sort is $O(n \log n)$ but gives full order.

■ QUICK REVISION

- Python heapq = min-heap; negate values for max-heap
- heapify = $O(n)$; push/pop = $O(\log n)$; peek = $O(1)$
- Top-K problems: maintain a heap of size K
- Median in stream: two heaps (max-heap lower half + min-heap upper half)
- Merge K sorted lists: $O(N \log k)$ with heap

Chapter 8: Graphs

8.1 Graph Representations

Representation	Space	Add Edge	Check Edge	Best For
Adjacency Matrix	$O(V^2)$	$O(1)$	$O(1)$	Dense graphs
Adjacency List	$O(V+E)$	$O(1)$	$O(\text{degree})$	Sparse graphs (most problems)
Edge List	$O(E)$	$O(1)$	$O(E)$	Kruskal's MST

```
# Adjacency List (most common in interviews)
from collections import defaultdict, deque
```

```
graph = defaultdict(list)
edges = [(0,1), (0,2), (1,3), (2,3), (3,4)]
for u, v in edges:
    graph[u].append(v)
    graph[v].append(u) # undirected
```

8.2 BFS – Breadth First Search

```
def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for nei in graph[node]:
            if nei not in visited:
                visited.add(nei)
                queue.append(nei)
    return order #  $O(V + E)$ 
```

8.3 DFS – Depth First Search

```
def dfs(graph, node, visited=None):
    if visited is None: visited = set()
    visited.add(node)
    for nei in graph[node]:
        if nei not in visited:
            dfs(graph, nei, visited)
    return visited #  $O(V + E)$ 
```

8.4 Topological Sort (Kahn's Algorithm)

```
def topo_sort(n, edges):
    graph = defaultdict(list); in_deg = [0]*n
    for u, v in edges:
        graph[u].append(v); in_deg[v] += 1
    q = deque(i for i in range(n) if in_deg[i]==0)
    order = []
    while q:
        u = q.popleft(); order.append(u)
        for v in graph[u]:
            in_deg[v] -= 1
```

```

        if in_deg[v] == 0: q.append(v)
    return order if len(order)==n else [] # empty = cycle exists

```

8.5 Dijkstra's Shortest Path

```

import heapq

def dijkstra(graph, src, n):
    dist = [float('inf')] * n; dist[src] = 0
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]: continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(heap, (dist[v], v))
    return dist # O((V + E) log V)

```

■ Most Asked Interview Questions

Q: BFS vs DFS – when to use which?

A: BFS: shortest path in unweighted graphs, level-order processing. DFS: cycle detection, topological sort, finding connected components, backtracking.

Q: How to detect a cycle in a directed graph?

A: DFS with three states: WHITE (unvisited), GRAY (in-progress), BLACK (done). Back edge to a GRAY node = cycle.

Q: What is the difference between Dijkstra and BFS for shortest path?

A: BFS works on unweighted graphs (all edges = 1). Dijkstra handles weighted graphs with non-negative weights using a min-heap.

Q: What is a minimum spanning tree?

A: A spanning tree with minimum total edge weight. Algorithms: Kruskal's (sort edges, use Union-Find) and Prim's (greedy, use heap). Both $O(E \log V)$.

■ QUICK REVISION

- BFS = queue, shortest path (unweighted), $O(V+E)$
- DFS = recursion/stack, cycle detection, topological sort, $O(V+E)$
- Topological Sort: only for DAGs (Directed Acyclic Graphs)
- Dijkstra: weighted shortest path, $O((V+E) \log V)$, no negative edges
- Adjacency list preferred for sparse graphs (most interview problems)

Chapter 9: Hashing

9.1 How a Hash Table Works

A hash table maps keys to values using a hash function. The hash function converts a key into an index in an array. Average operations are $O(1)$; worst case $O(n)$ due to collisions.

Operation	Average	Worst Case
Search	$O(1)$	$O(n)$
Insert	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$

9.2 Collision Handling

Method	Description	Pro	Con
Chaining	Each bucket holds a linked list	Simple, never overflows	Extra memory, cache unfriendly
Open Addressing	Probe next empty slot	Cache friendly, less memory	Clustering, load factor limits
Linear Probing	Check next slot one-by-one	Simple probing	Primary clustering
Quadratic	Probe at i^2 steps	Reduces clustering	Secondary clustering
Double Hashing	Second hash for step size	Best distribution	Two hash functions needed

```
# Common HashMap patterns
from collections import Counter, defaultdict

# Frequency count
freq = Counter("abracadabra")    # {'a':5, 'b':2, 'r':2, 'c':1, 'd':1}

# Group anagrams
def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        groups[tuple(sorted(w))].append(w)
    return list(groups.values())

# Two Sum
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen: return [seen[target-n], i]
        seen[n] = i
```

■ Most Asked Interview Questions

Q: What is the load factor in a hash table?

A: Load factor = n/m where n = number of entries, m = number of buckets. Most implementations resize (rehash) when load factor > 0.75 .

Q: Difference between HashMap and HashSet?

A: HashMap stores key-value pairs; HashSet stores only keys (backed by a HashMap with dummy values). Both use hashing.

Q: How does Python dict handle collisions?

A: Python dicts use open addressing with a pseudo-random probing sequence derived from the hash value.

■ **QUICK REVISION**

- HashMap: $O(1)$ average for get/put/delete
- Load factor > 0.75 triggers rehashing ($O(n)$ but amortized $O(1)$)
- Chaining vs Open Addressing: both valid, chaining more common in interviews
- Counter and defaultdict are built-in Python tools for hashing patterns
- Use frozenset or tuple as hashable keys for complex keys

Chapter 10: Recursion and Backtracking

10.1 Recursion Template

```
def solve(params):
    # Base case
    if base_condition: return base_value
    # Recursive case
    result = solve(smaller_problem)
    return combine(result, current)
```

10.2 N-Queens Problem

```
def solve_n_queens(n):
    results = []
    board = [['.'] * n for _ in range(n)]

    def is_safe(row, col):
        for i in range(row):
            if board[i][col] == 'Q': return False
            if col - (row - i) >= 0 and board[i][col - (row - i)] == 'Q': return False
            if col + (row - i) < n and board[i][col + (row - i)] == 'Q': return False
        return True

    def backtrack(row):
        if row == n:
            results.append(''.join(r for r in board))
            return
        for col in range(n):
            if is_safe(row, col):
                board[row][col] = 'Q'
                backtrack(row + 1)
                board[row][col] = '.' # undo (backtrack)

    backtrack(0)
    return results
```

10.3 Subsets, Permutations, Combinations

```
# Subsets (Power Set)
def subsets(nums):
    result = [[]]
    for n in nums:
        result += [s + [n] for s in result]
    return result

# Permutations
def permutations(nums):
    if len(nums) <= 1: return [nums]
    result = []
    for i, n in enumerate(nums):
        rest = nums[:i] + nums[i+1:]
        for perm in permutations(rest):
            result.append([n] + perm)
    return result

# Combinations of size k
```

```
def combinations(nums, k):
    result = []
    def backtrack(start, path):
        if len(path) == k: result.append(path[:]); return
        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i+1, path)
            path.pop()
    backtrack(0, [])
    return result
```

10.4 Word Search (Grid Backtracking)

```
def word_search(board, word):
    R, C = len(board), len(board[0])

    def dfs(r, c, idx):
        if idx == len(word): return True
        if r<0 or r>=R or c<0 or c>=C or board[r][c] != word[idx]: return False
        tmp = board[r][c]; board[r][c] = '#' # mark visited
        found = any(dfs(r+dr, c+dc, idx+1) for dr,dc in [(0,1),(0,-1),(1,0),(-1,0)])
        board[r][c] = tmp # restore
        return found

    return any(dfs(r,c,0) for r in range(R) for c in range(C))
```

■ Most Asked Interview Questions

Q: What is the time complexity of generating all subsets?

A: $O(2^n * n)$ — 2^n subsets, each taking $O(n)$ to copy.

Q: How do you avoid duplicates in combination/subset problems?

A: Sort the input. Skip `nums[i]` if `i > start` and `nums[i] == nums[i-1]` in the backtracking loop.

Q: What is the difference between backtracking and brute force?

A: Backtracking prunes branches early (abandons partial solutions that cannot lead to a valid answer), making it much faster than pure brute force.

■ QUICK REVISION

- Backtracking = make choice → recurse → undo choice
- Always handle base case first in recursive functions
- N-Queens: check column, and both diagonals for each placement
- Subsets: $O(2^n)$, Permutations: $O(n!)$, Combinations(n,k): $O(C(n,k))$
- Mark visited cells with special char (#) in grid DFS; restore after

Chapter 11: Dynamic Programming

11.1 What is DP?

Dynamic Programming breaks a problem into overlapping sub-problems and stores the results to avoid recomputation. Two approaches: **Top-Down (Memoization)** = recursion + cache, and **Bottom-Up (Tabulation)** = iterative, fill table.

11.2 Fibonacci – Memoization vs Tabulation

```
# Naive Recursion:  $O(2^n)$ 
def fib_naive(n):
    if n <= 1: return n
    return fib_naive(n-1) + fib_naive(n-2)

# Memoization (Top-Down):  $O(n)$  time,  $O(n)$  space
from functools import lru_cache
@lru_cache(maxsize=None)
def fib_memo(n):
    if n <= 1: return n
    return fib_memo(n-1) + fib_memo(n-2)

# Tabulation (Bottom-Up):  $O(n)$  time,  $O(1)$  space
def fib_tab(n):
    if n <= 1: return n
    a, b = 0, 1
    for _ in range(2, n+1): a, b = b, a+b
    return b
```

11.3 0/1 Knapsack

```
def knapsack(weights, values, capacity):
    n = len(weights)
    # dp[i][w] = max value using first i items with capacity w
    dp = [[0]*(capacity+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(capacity+1):
            dp[i][w] = dp[i-1][w] # don't take item i
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w], dp[i-1][w-weights[i-1]] + values[i-1])
    return dp[n][capacity] #  $O(n * \text{capacity})$ 
```

11.4 Longest Common Subsequence (LCS)

```
def lcs(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(1, m+1):
        for j in range(1, n+1):
            if s1[i-1] == s2[j-1]: dp[i][j] = dp[i-1][j-1] + 1
            else: dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n] #  $O(m*n)$  time and space
```

11.5 Longest Increasing Subsequence (LIS)

```
def lis(nums):
    #  $O(n^2)$  DP
    n = len(nums); dp = [1]*n
```

```

for i in range(1, n):
    for j in range(i):
        if nums[j] < nums[i]: dp[i] = max(dp[i], dp[j]+1)
return max(dp)

def lis_binary(nums):
    # O(n log n) with patience sort
    import bisect
    tails = []
    for n in nums:
        pos = bisect.bisect_left(tails, n)
        if pos == len(tails): tails.append(n)
        else: tails[pos] = n
    return len(tails)

```

11.6 Edit Distance (Levenshtein)

```

def edit_distance(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(m+1): dp[i][0] = i # delete all
    for j in range(n+1): dp[0][j] = j # insert all
    for i in range(1, m+1):
        for j in range(1, n+1):
            if s1[i-1] == s2[j-1]: dp[i][j] = dp[i-1][j-1]
            else: dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
    return dp[m][n]

```

■ Most Asked Interview Questions

Q: How do you identify if a problem can be solved with DP?

A: Look for: optimal substructure (optimal solution contains optimal sub-solutions) AND overlapping sub-problems (same sub-problems solved multiple times).

Q: Coin Change: minimum coins to make amount n.

A: $dp[i] = \text{min coins for amount } i$. $dp[0]=0$. For each coin, $dp[i] = \min(dp[i], dp[i-\text{coin}]+1)$. $O(\text{amount} * \text{coins})$.

Q: What is the difference between DP and Greedy?

A: DP considers all possibilities and picks the best; Greedy makes the locally optimal choice at each step. Greedy is faster but not always correct.

Q: Longest Palindromic Subsequence.

A: $LPS(s) = LCS(s, \text{reverse}(s))$. $O(n^2)$ time and space.

■ QUICK REVISION

- DP = Optimal Substructure + Overlapping Sub-problems
- Top-Down (memo) easier to write; Bottom-Up (tabulation) space-optimizable
- Knapsack pattern: $dp[i][w]$ — include or exclude item
- LCS/Edit Distance: 2D DP table, $O(m*n)$
- LIS: $O(n^2)$ DP or $O(n \log n)$ with binary search + patience sort

Chapter 12: Greedy Algorithms

12.1 Greedy Strategy

A greedy algorithm makes the locally optimal choice at each step, hoping to find the global optimum. It works when the problem has the **greedy choice property** and **optimal substructure**.

12.2 Activity Selection Problem

```
# Sort by finish time, greedily pick non-overlapping activities
def activity_selection(start, finish):
    activities = sorted(zip(finish, start))
    selected = [activities[0]]
    last_finish = activities[0][0]
    for f, s in activities[1:]:
        if s >= last_finish:
            selected.append((f, s))
            last_finish = f
    return selected    # O(n log n)
```

12.3 Fractional Knapsack

```
def fractional_knapsack(weights, values, capacity):
    items = sorted(zip(values, weights), key=lambda x: x[0]/x[1], reverse=True)
    total = 0
    for v, w in items:
        if capacity >= w: total += v; capacity -= w
        else: total += v * (capacity/w); break
    return total    # O(n log n)
```

12.4 Meeting Rooms – Minimum Rooms Required

```
import heapq
def min_meeting_rooms(intervals):
    if not intervals: return 0
    intervals.sort()
    rooms = [intervals[0][1]]    # min-heap of end times
    for start, end in intervals[1:]:
        if rooms[0] <= start: heapq.heapreplace(rooms, end)
        else: heapq.heappush(rooms, end)
    return len(rooms)
```

■ Most Asked Interview Questions

Q: When does greedy fail?

A: When local optimum doesn't lead to global optimum (e.g., 0/1 Knapsack). Always verify with a proof (exchange argument) or counter-example.

Q: Jump Game: can you reach the last index?

A: Greedily track the farthest index reachable. If current index > farthest, return False. $O(n)$.

Q: Minimum number of platforms needed at a train station.

A: Sort arrivals and departures. Use two pointers: if next event is arrival, platforms++; if departure, platforms--. Track max.

■ QUICK REVISION

- Greedy = sort + make locally optimal choice + prove it's globally optimal

- Activity Selection: sort by finish time
- Fractional Knapsack: sort by value/weight ratio
- Meeting Rooms: sort by start, use min-heap of end times
- Greedy vs DP: verify with exchange argument or counter-example

Chapter 13: Fix-the-Code Round

Fix-the-code rounds test your ability to read, understand, and debug existing code. You are given broken code and must identify and fix the bug(s).

Common Bug Categories

Bug Type	Description	Example
Off-by-one	Wrong loop bounds	range(n) vs range(n+1)
Wrong base case	Recursion never terminates or returns wrong value	if n==0: return 1 (should be 0)
Integer overflow	Large values exceed int range	Use // and avoid multiplying large nums
Null/None check	Accessing member of None object	if not node: return before node.val
Wrong comparison	Using = instead of ==	if x = 5 → SyntaxError in Python, logical error in Java
Mutating while iterating	Changing a list during iteration	Use a copy or iterate backwards
Wrong return	Missing return in branch	if condition: compute (but forget return)

Bug Fix Example 1 – Binary Search

```
# BUGGY:
def binary_search_bug(arr, target):
    l, r = 0, len(arr)          # BUG: should be len(arr)-1
    while l < r:                # BUG: should be l <= r
        mid = (l + r) / 2       # BUG: should be // (integer division)
        if arr[mid] == target: return mid
        elif arr[mid] < target: l = mid    # BUG: should be mid + 1
        else: r = mid           # BUG: should be mid - 1

# FIXED:
def binary_search_fix(arr, target):
    l, r = 0, len(arr) - 1
    while l <= r:
        mid = l + (r - l) // 2
        if arr[mid] == target: return mid
        elif arr[mid] < target: l = mid + 1
        else: r = mid - 1
    return -1
```

Bug Fix Example 2 – Linked List Cycle

```
# BUGGY:
def has_cycle_bug(head):
    slow = head; fast = head.next    # BUG: NullPointerException if head is None
    while slow != fast:
        if fast is None: return False # BUG: fast.next not checked
        slow = slow.next
        fast = fast.next.next        # BUG: fast could be None here
    return True

# FIXED:
def has_cycle_fix(head):
```

```
slow = fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
    if slow == fast: return True
return False
```

■ **TIP:** Read the entire function before fixing. Understand what it is SUPPOSED to do, then trace through with a simple example to find where it breaks.

■ **COMMON MISTAKE:** Always check for off-by-one errors first — they account for 40%+ of fix-the-code bugs in interviews.

■ Most Asked Interview Questions

Q: What is the first thing you do when given buggy code?

A: Understand the intent. Read the function signature and any comments. Then trace through a simple example to find where the actual output diverges from expected.

Q: How do you debug infinite loops?

A: Check loop termination condition. Verify that loop variables are being modified in each iteration. Check for cases where condition can never become false.

Q: What is a segmentation fault / NullPointerException?

A: Accessing memory/object that doesn't exist. In tree/linked list code: always check if node is None before accessing node.val or node.next.

■ QUICK REVISION

- Off-by-one: most common — check loop bounds carefully
- None/Null checks: before accessing .next, .val, .left, .right
- Integer division: use // in Python for indices
- Return values: check every branch returns correct type
- Test with: empty input, single element, sorted/reverse-sorted, duplicates

SECTION 2: FULL STACK DEVELOPMENT

Chapter 14: JavaScript Basics

14.1 Variables & Scoping

Keyword	Scope	Re-declare	Re-assign	Hoisted
var	Function	Yes	Yes	Yes (undefined)
let	Block	No	Yes	No (TDZ)
const	Block	No	No	No (TDZ)

■ NOTE: TDZ = Temporal Dead Zone: accessing let/const before declaration throws ReferenceError.

14.2 Closures

```
function counter() {
  let count = 0;
  return {
    increment: () => ++count,
    decrement: () => --count,
    value:    () => count
  };
}
const c = counter();
c.increment(); c.increment();
console.log(c.value()); // 2
// 'count' is enclosed in the returned object's scope
```

14.3 Promises & Async/Await

```
// Promise chain
fetch('/api/data')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));

// Async/Await (syntactic sugar over promises)
async function getData() {
  try {
    const res = await fetch('/api/data');
    const data = await res.json();
    return data;
  } catch (err) {
    console.error(err);
  }
}

// Run promises in parallel
const [users, posts] = await Promise.all([
  fetch('/api/users').then(r => r.json()),
  fetch('/api/posts').then(r => r.json()),
]);
```

14.4 ES6+ Features

- **Arrow functions:** `const add = (a,b) => a+b;`
- **Destructuring:** `const {name, age} = person;`
- **Spread/Rest:** `const newArr = [...arr, 4];`
- **Template literals:** ``Hello ${name}!``
- **Optional chaining:** `user?.address?.city`
- **Nullish coalescing:** `value ?? defaultValue`
- **Modules:** `import/export` (ESM)

■ Most Asked Interview Questions

Q: What is the difference between `==` and `===` in JavaScript?

A: `==` performs type coercion (`5 == '5'` is true). `===` is strict equality without coercion (`5 === '5'` is false). Always use `===`.

Q: Explain event delegation.

A: Instead of adding listeners to many child elements, add one listener to the parent. Use `event.target` to identify which child was clicked. More memory-efficient.

Q: What is the difference between null and undefined?

A: `undefined` = variable declared but not assigned. `null` = intentional absence of value (explicitly set). `typeof null === 'object'` is a historical JS bug.

Q: Explain the JavaScript event loop.

A: JS is single-threaded. Call stack runs sync code. Web APIs handle async tasks. Callback queue holds resolved callbacks. Event loop moves callbacks to call stack when stack is empty. Microtasks (Promises) run before macrotasks (`setTimeout`).

■ QUICK REVISION

- `var`: function-scoped, hoisted; `let/const`: block-scoped, TDZ
- Closure: inner function remembers outer function's variables
- Promise states: pending → fulfilled/rejected
- `async/await` is syntactic sugar for `.then()/.catch()`
- Event loop: call stack → microtask queue → macrotask queue

Chapter 15: Node.js Basics

15.1 The Event Loop

Node.js is single-threaded but non-blocking, using an event-driven architecture. The Event Loop has phases: **timers** → **I/O callbacks** → **idle/prepare** → **poll** → **check** → **close callbacks**.

```
// Non-blocking I/O example
const fs = require('fs');
console.log('Start');
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log('File read:', data);
});
console.log('End');
// Output order: Start → End → File read
```

15.2 CommonJS Modules

```
// math.js
const add = (a, b) => a + b;
module.exports = { add };

// app.js
const { add } = require('./math');
console.log(add(2, 3)); // 5
```

15.3 Express Server Setup

```
const express = require('express');
const app = express();
app.use(express.json());

app.get('/api/users', (req, res) => {
  res.json({ users: [] });
});

app.listen(3000, () => console.log('Server on port 3000'));
```

15.4 JWT Authentication Flow

```
const jwt = require('jsonwebtoken');
const SECRET = process.env.JWT_SECRET;

// Generate token
const token = jwt.sign({ userId: user.id }, SECRET, { expiresIn: '1h' });

// Verify token (middleware)
function authMiddleware(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'No token' });
  try {
    req.user = jwt.verify(token, SECRET);
    next();
  } catch {
    res.status(401).json({ error: 'Invalid token' });
  }
}
```

■ Most Asked Interview Questions

Q: What is the difference between `process.nextTick` and `setImmediate`?

A: `process.nextTick` fires before any I/O events in the same phase (before poll). `setImmediate` fires in the check phase (after I/O). `nextTick` has higher priority.

Q: How does Node.js handle concurrency if it's single-threaded?

A: Through non-blocking I/O with the libuv thread pool (for file I/O, DNS, crypto) and the event loop. Network I/O uses the OS kernel directly.

Q: What is middleware in Express?

A: Functions with (req, res, next) signature that execute in sequence. Used for logging, authentication, parsing request body, error handling, etc.

■ QUICK REVISION

- Node.js: single-threaded, non-blocking I/O via event loop
- Event loop phases: timers → I/O → poll → check → close
- `require()` = sync; `fs.readFile()` = async
- JWT: sign on login, verify on every protected route
- `process.nextTick` > `Promise.then` > `setImmediate` > `setTimeout` priority

Chapter 16: React.js – Basic to Advanced

16.1 Core Concepts

- **JSX:** JavaScript + XML syntax. Compiled to `React.createElement` calls.
- **Components:** Functions (or classes) that return JSX. Reusable UI pieces.
- **Props:** Read-only data passed from parent to child.
- **State:** Mutable data owned by a component. Changes trigger re-render.
- **Virtual DOM:** React's in-memory representation. Diffed against real DOM for efficient updates.

16.2 Hooks

```
import React, { useState, useEffect, useCallback, useMemo, useRef } from 'react';

function UserCard({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const renderCount = useRef(0);

  // Runs after render; fetches user data when userId changes
  useEffect(() => {
    setLoading(true);
    fetch(`/api/users/${userId}`)
      .then(r => r.json())
      .then(data => { setUser(data); setLoading(false); });
    return () => { /* cleanup (cancel request, clear timers) */ };
  }, [userId]); // dependency array

  // Memoize expensive calculation
  const fullName = useMemo(() =>
    user ? `${user.firstName} ${user.lastName}` : '',
    [user]);

  // Stable callback reference (avoids unnecessary child re-renders)
  const handleClick = useCallback(() => {
    console.log('Clicked', user.id);
  }, [user]);

  renderCount.current++;
  if (loading) return <p>Loading...</p>;
  return <div onClick={handleClick}>{fullName} (renders: {renderCount.current})</div>;
}
```

16.3 Context API

```
const ThemeContext = React.createContext('light');

function App() {
  const [theme, setTheme] = useState('light');
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
```

```

const { theme, setTheme } = React.useContext(ThemeContext);
return <button onClick={() => setTheme(t => t==='light'? 'dark': 'light')}>
  Current: {theme}
</button>;
}

```

16.4 Performance Optimization

Tool	Purpose	When to Use
React.memo	Skip re-render if props unchanged	Expensive pure components
useMemo	Memoize computed value	Expensive calculations
useCallback	Memoize function reference	Pass callbacks to children
Code Splitting	Load components lazily	Large apps, route-based
Virtualization	Render only visible rows	Long lists (react-window)

■ Most Asked Interview Questions

Q: What is the difference between controlled and uncontrolled components?

A: Controlled: form data managed by React state (onChange + value). Uncontrolled: form data managed by DOM (useRef to access).

Q: When would you use Redux over Context API?

A: Redux for complex global state with many updates, time-travel debugging, or middleware (like redux-thunk for async). Context for simple global state (theme, auth).

Q: What happens when you call setState?

A: React schedules a re-render. Multiple setState calls in the same handler are batched. The component re-renders with new state. Children re-render unless memoized.

Q: Explain React's reconciliation algorithm.

A: React diffs the new Virtual DOM tree against the previous one. It uses the key prop to identify list items. Elements of different types → unmount old, mount new.

■ QUICK REVISION

- useState: local state; useEffect: side effects; useRef: mutable ref
- useMemo: cache computed value; useCallback: cache function reference
- Context API: avoid prop drilling for global state
- React.memo + useCallback/useMemo to prevent unnecessary re-renders
- Keys in lists must be stable, unique, and not array index (if list reorders)

Chapter 17: Express.js

17.1 Routing

```
const router = require('express').Router();

// CRUD for /api/products
router.get('/', listProducts);
router.get('/:id', getProduct);
router.post('/', createProduct);
router.put('/:id', updateProduct);
router.delete('/:id', deleteProduct);

// Route params and query strings
router.get('/search', (req, res) => {
  const { q, page = 1, limit = 10 } = req.query;
  res.json({ query: q, page, limit });
});
```

17.2 Middleware Stack

```
// Global middleware
app.use(express.json());
app.use(cors());
app.use(morgan('dev'));

// Auth middleware
const authenticate = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'Unauthorized' });
  req.user = jwt.verify(token, SECRET);
  next();
};

// Protected route
app.get('/api/profile', authenticate, (req, res) => {
  res.json(req.user);
});
```

17.3 Error Handling Middleware

```
// Must have 4 params; placed LAST
app.use((err, req, res, next) => {
  const status = err.status || 500;
  const message = err.message || 'Internal Server Error';
  console.error(err.stack);
  res.status(status).json({ error: message });
});

// Throwing errors from route handlers
app.get('/risky', async (req, res, next) => {
  try {
    const data = await riskyOperation();
    res.json(data);
  } catch (err) {
    next(err); // passes to error handler
  }
});
```

```
});
```

■ Most Asked Interview Questions

Q: What is the order of middleware execution?

A: Middleware runs in the order it is defined with `app.use()`. Routes also count as middleware. Error-handling middleware (4 params) runs only when `next(err)` is called.

Q: How do you handle CORS in Express?

A: Use the cors npm package: `app.use(cors({ origin: 'https://frontend.com' })))`. Or manually set headers: `res.header('Access-Control-Allow-Origin', '*')`.

Q: What is the difference between `app.use()` and `app.get()`?

A: `app.use()` matches any HTTP method and can match path prefixes. `app.get()` only matches GET requests on exact paths.

■ QUICK REVISION

- Middleware: functions with (req, res, next). Call `next()` to continue chain.
- Error middleware: must have 4 params (err, req, res, next)
- Route params: `req.params.id`. Query strings: `req.query.name`. Body: `req.body`
- `cors()` package handles cross-origin headers automatically
- async route handlers need `try/catch` and `next(err)` for error propagation

Chapter 18: Spring Boot

18.1 Core Concepts

Concept	Description
IoC Container	Spring manages object creation and lifecycle
Dependency Injection	Objects receive their dependencies from the container, not create them
Spring Bean	Any object managed by the Spring container
@Component	Marks class as a Spring-managed component
@Service	@Component for service layer (business logic)
@Repository	@Component for data access layer (DAOs)
@Controller	@Component for web layer (handles HTTP requests)
@RestController	@Controller + @ResponseBody (returns JSON directly)
@Autowired	Injects a dependency automatically by type

18.2 REST API – Full Example

```
// Entity
@Entity
@Table(name = "products")
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false) private String name;
    private BigDecimal price;
    // getters, setters, constructors
}

// Repository
public interface ProductRepository extends JpaRepository<Product, Long> {
    List<Product> findByNameContaining(String keyword);
}

// Service
@Service
public class ProductService {
    @Autowired private ProductRepository repo;

    public List<Product> getAllProducts() { return repo.findAll(); }
    public Product getById(Long id) { return repo.findById(id).orElseThrow(); }
    public Product create(Product p) { return repo.save(p); }
    public void delete(Long id) { repo.deleteById(id); }
}

// Controller
@RestController
@RequestMapping("/api/products")
public class ProductController {
    @Autowired private ProductService service;
    @GetMapping public List<Product> list() { return service.getAllProducts(); }
```

```

    @GetMapping("/{id}")    public Product    get(@PathVariable Long id) { return service.getById(id);
    @PostMapping            public Product    create(@RequestBody @Valid Product p) { return service.
    @DeleteMapping("/{id}") public void      delete(@PathVariable Long id) { service.delete(id); }
}

```

18.3 Spring Security + JWT

```

// Security Config (Spring Boot 3+)
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }
}

```

18.4 Microservices Overview

- **API Gateway:** Single entry point (Spring Cloud Gateway). Routes requests, handles auth, rate limiting.
- **Service Discovery:** Eureka Server — services register themselves; clients discover by name.
- **Config Server:** Centralized configuration for all services.
- **Load Balancing:** Ribbon (client-side) or Spring Cloud LoadBalancer.
- **Circuit Breaker:** Resilience4j — stops cascading failures when a service is down.
- **Inter-service Communication:** REST (RestTemplate/WebClient) or async messaging (Kafka/RabbitMQ).

■ Most Asked Interview Questions

Q: What is the difference between @Component, @Service, and @Repository?

A: All are @Component specializations (Spring-managed beans). @Service = business logic.
@Repository = data access (also translates DB exceptions to Spring exceptions). @Component = generic.

Q: What is Spring Data JPA?

A: Abstraction over JPA/Hibernate. Extend JpaRepository for auto-generated CRUD + custom queries by method name (e.g., findByEmail).

Q: Explain the difference between @Transactional and manual transaction management.

A: @Transactional: declarative, Spring handles begin/commit/rollback via AOP. Manual: programmatic with TransactionTemplate. Declarative is preferred for simplicity.

Q: What is the difference between monolith and microservices?

A: Monolith: single deployable unit; simpler but hard to scale parts independently. Microservices: independent services; better scaling and fault isolation but complex infra (service discovery, distributed tracing, etc.).

■ QUICK REVISION

- @RestController = @Controller + @ResponseBody — returns JSON
- Spring Bean lifecycle: instantiate → inject → @PostConstruct → use → @PreDestroy
- JpaRepository gives: save, findById, findAll, delete — for free

- JWT filter: extract token → validate → set SecurityContext → chain.doFilter
- Microservices: each service owns its DB; communicate via REST or messaging

Chapter 19: Databases & Advanced DBMS

19.1 Normalization

Normal Form	Rule	Violates If
1NF	Atomic columns, no repeating groups	Column stores comma-separated values
2NF	1NF + no partial dependency on composite PK	Non-key column depends on part of PK
3NF	2NF + no transitive dependency	Non-key column depends on another non-key
BCNF	Every determinant is a candidate key	A non-candidate-key determines a column
4NF	BCNF + no multi-valued dependencies	Two independent multi-valued facts in one table

19.2 ACID Properties

Property	Meaning	Example
Atomicity	All or nothing — transaction fully succeeds or fully rolls back	Bank transfer: debit+credit both succeed or neither
Consistency	DB moves from one valid state to another	Constraints, triggers, cascades maintain integrity
Isolation	Concurrent transactions don't interfere	Uncommitted read of T1 not visible to T2
Durability	Committed data persists even after crash	Written to disk/WAL log before commit returns

19.3 SQL Joins

Join Type	Returns
INNER JOIN	Rows with matching values in BOTH tables
LEFT JOIN	All rows from left table + matching right (NULL if no match)
RIGHT JOIN	All rows from right table + matching left (NULL if no match)
FULL OUTER	All rows from both tables (NULL where no match)
CROSS JOIN	Cartesian product — every row paired with every row
SELF JOIN	Table joined with itself (e.g., employee → manager)

19.4 Indexing

- **B-Tree Index:** Default in most DBs. Good for equality, range, ORDER BY.
- **Hash Index:** O(1) lookup for equality only. No range queries.
- **Composite Index:** Index on (a, b, c). Useful for queries on prefix (a) or (a,b).
- **Covering Index:** Index contains all columns needed — no heap fetch.
- **When NOT to index:** Small tables, columns with low cardinality, write-heavy tables.

19.5 Advanced SQL Queries (Selected)

-- Window Functions

```

SELECT name, department, salary,
       RANK()      OVER (PARTITION BY department ORDER BY salary DESC) AS dept_rank,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS global_rank,
       LAG(salary) OVER (PARTITION BY department ORDER BY salary) AS prev_salary
FROM employees;

-- CTE (Common Table Expression)
WITH dept_avg AS (
    SELECT department, AVG(salary) AS avg_sal
    FROM employees GROUP BY department
)
SELECT e.name, e.salary, d.avg_sal
FROM employees e JOIN dept_avg d ON e.department = d.department
WHERE e.salary > d.avg_sal;

-- Find Nth highest salary
WITH ranked AS (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
)
SELECT salary FROM ranked WHERE rnk = 3;    -- 3rd highest

-- Detect duplicates
SELECT email, COUNT(*) AS cnt
FROM users
GROUP BY email HAVING COUNT(*) > 1;

-- Recursive CTE (org chart)
WITH RECURSIVE org AS (
    SELECT id, name, manager_id FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.id, e.name, e.manager_id
    FROM employees e JOIN org ON e.manager_id = org.id
)
SELECT * FROM org;

```

■ Most Asked Interview Questions

Q: What is the difference between WHERE and HAVING?

A: WHERE filters rows BEFORE grouping. HAVING filters groups AFTER GROUP BY. You can't use aggregate functions in WHERE.

Q: Explain EXPLAIN ANALYZE.

A: Shows the query execution plan with actual vs estimated rows, costs, and time. Look for sequential scans on large tables — they need an index.

Q: What are stored procedures vs functions?

A: Stored procedures: can execute DML, can't be used in SELECT. Functions: must return a value, can be used in SELECT, typically no side effects.

Q: What is a deadlock and how do you prevent it?

A: Two transactions each wait for the other to release a lock. Prevention: always acquire locks in the same order, keep transactions short, use SELECT FOR UPDATE SKIP LOCKED.

■ QUICK REVISION

- Normalization: 1NF=atomic, 2NF=no partial dep, 3NF=no transitive dep
- ACID: Atomicity, Consistency, Isolation, Durability
- B-Tree index: range + equality; Hash: equality only
- Window functions: RANK, ROW_NUMBER, LAG/LEAD, SUM OVER PARTITION

- CTE (WITH clause) improves readability and enables recursion

Chapter 20: Redis

20.1 What is Redis?

Redis (Remote Dictionary Server) is an in-memory key-value store. It supports rich data structures and is used for caching, session storage, pub/sub messaging, rate limiting, and leaderboards.

20.2 Data Structures

Type	Commands	Use Case
String	SET, GET, INCR, EXPIRE	Cache values, counters, session tokens
List	LPUSH, RPUSH, LRANGE, LLEN	Message queues, activity feeds
Set	SADD, SMEMBERS, SINTER	Tags, unique visitors, friend lists
Sorted Set	ZADD, ZRANGE, ZRANK	Leaderboards, priority queues
Hash	HSET, HGET, HGETALL	User objects, shopping carts
Bitmap	SETBIT, BITCOUNT	Daily active user tracking
HyperLogLog	PFADD, PFCOUNT	Approximate unique count

```
import redis
r = redis.Redis(host='localhost', port=6379, decode_responses=True)

# String with TTL
r.set('user:1:name', 'Alice', ex=3600) # expires in 1 hour
print(r.get('user:1:name'))           # 'Alice'

# Sorted Set - Leaderboard
r.zadd('leaderboard', {'Alice': 100, 'Bob': 95, 'Carol': 110})
print(r.zrevrange('leaderboard', 0, 2, withscores=True))
# [('Carol', 110.0), ('Alice', 100.0), ('Bob', 95.0)]

# Cache-Aside Pattern
def get_user(user_id):
    cache_key = f'user:{user_id}'
    cached = r.get(cache_key)
    if cached: return json.loads(cached) # cache hit
    user = db.query(f'SELECT * FROM users WHERE id={user_id}')
    r.set(cache_key, json.dumps(user), ex=300) # cache for 5 min
    return user
```

20.3 Cache Invalidation Strategies

Strategy	Description	Best For
TTL / Expiry	Keys auto-expire after a set time	Data that changes on a schedule
Cache-Aside	App checks cache; on miss loads from DB	Read-heavy workloads
Write-Through	Write to cache AND DB synchronously	Strong consistency needed
Write-Behind	Write to cache; async flush to DB	Write-heavy, tolerate eventual consistency
Event-driven	Invalidate cache on DB change event	Complex dependencies

■ Most Asked Interview Questions

Q: What is Redis Persistence?

A: RDB (snapshot): periodic point-in-time snapshots. AOF (Append-Only File): logs every write command. AOF is safer (less data loss); RDB is faster for restarts.

Q: How does Redis handle concurrency?

A: Redis is single-threaded for command processing, so commands are atomic. Use MULTI/EXEC for transactions. Lua scripts for atomic complex operations.

Q: What is a Redis cluster?

A: Data is sharded across multiple nodes using consistent hashing (16384 hash slots). Provides horizontal scaling and high availability with replicas.

■ QUICK REVISION

- Redis: in-memory, sub-millisecond reads, volatile by default
- Key data structures: String, List, Set, Sorted Set, Hash
- Cache-aside: check cache → miss → load DB → write cache
- TTL: always set expiry to avoid stale data filling memory
- Redis is single-threaded → all commands are atomic

Chapter 21: RabbitMQ

21.1 Core Concepts

Concept	Description
Producer	Application that sends messages
Consumer	Application that receives messages
Queue	Buffer that stores messages until consumed
Exchange	Receives messages from producer and routes to queues based on rules
Binding	Link between exchange and queue with optional routing key
Channel	Lightweight connection within a TCP connection (cheaper to create)
Virtual Host	Logical grouping/namespace for exchanges, queues, and bindings

21.2 Exchange Types

Type	Routing Rule	Use Case
Direct	Routes to queues where binding key == routing key	Task queues, load balancing
Fanout	Routes to ALL bound queues (ignores routing key)	Broadcast, pub/sub
Topic	Wildcard matching (* = one word, # = zero or more)	News feeds, filtered events
Headers	Routes based on message header attributes	Complex routing rules

```
import pika, json

# Producer
connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
channel = connection.channel()
channel.queue_declare(queue='tasks', durable=True)
channel.basic_publish(
    exchange='', routing_key='tasks',
    body=json.dumps({'task': 'send_email', 'to': 'user@example.com'}),
    properties=pika.BasicProperties(delivery_mode=2) # persistent
)
connection.close()

# Consumer
def callback(ch, method, properties, body):
    task = json.loads(body)
    print(f"Processing: {task}")
    ch.basic_ack(delivery_tag=method.delivery_tag) # acknowledge

channel.basic_qos(prefetch_count=1) # fair dispatch
channel.basic_consume(queue='tasks', on_message_callback=callback)
channel.start_consuming()
```

■ Most Asked Interview Questions

Q: What is the difference between RabbitMQ and Kafka?

A: RabbitMQ: smart broker, push-based, messages deleted after ACK, good for task queues. Kafka: dumb broker, pull-based, messages retained for a period, great for event streaming and replay.

Q: What is message acknowledgment?

A: Consumer sends ACK after processing. If consumer dies without ACK, RabbitMQ requeues the message. Prevents message loss.

Q: What is a Dead Letter Exchange (DLX)?

A: Messages that are rejected, expired, or exceed max retry count are routed to a DLX for inspection or reprocessing.

■ **QUICK REVISION**

- Exchange routes; Queue stores; Consumer processes
- Fanout = broadcast; Direct = routing key match; Topic = wildcard
- `delivery_mode=2` → message persists to disk across restarts
- `basic_ack` after processing; no ack = requeued on consumer crash
- Dead Letter Exchange for failed/expired messages

Chapter 22: Docker

22.1 Core Concepts

Term	Description
Image	Read-only template with app + dependencies + config
Container	Running instance of an image (isolated process)
Dockerfile	Script of instructions to build an image
Registry	Repository of images (Docker Hub, ECR, GCR)
Volume	Persistent storage shared between container and host
Network	Virtual network for containers to communicate

22.2 Dockerfile – Spring Boot App

```
# Multi-stage build (smaller final image)
FROM maven:3.9-eclipse-temurin-21 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

22.3 Docker Compose – Full Stack

```
version: '3.8'
services:
  backend:
    build: ./backend
    ports: ["8080:8080"]
    environment:
      - SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/mydb
      - SPRING_DATASOURCE_PASSWORD=secret
    depends_on: [db, redis]

  frontend:
    build: ./frontend
    ports: ["3000:80"]
    depends_on: [backend]

  db:
    image: postgres:16-alpine
    environment: { POSTGRES_DB: mydb, POSTGRES_PASSWORD: secret }
    volumes: [pgdata:/var/lib/postgresql/data]

  redis:
    image: redis:7-alpine
```

volumes:
pgdata:

22.4 Common Docker Commands

Command	Description
<code>docker build -t myapp:1.0 .</code>	Build image from Dockerfile in current dir
<code>docker run -p 8080:8080 myapp:1.0</code>	Run container, map host:container port
<code>docker ps</code>	List running containers
<code>docker logs</code>	View container logs
<code>docker exec -it sh</code>	Open shell inside running container
<code>docker-compose up -d</code>	Start all services in background
<code>docker-compose down -v</code>	Stop and remove containers + volumes

■ Most Asked Interview Questions

Q: What is the difference between Docker and a Virtual Machine?

A: VMs virtualize hardware (full OS per VM). Docker virtualizes only the OS userspace (containers share host kernel). Docker is faster, lighter, and more portable.

Q: What is a multi-stage Docker build?

A: Uses multiple FROM instructions. Earlier stages build/compile the app; only the final stage (smaller runtime image) is shipped. Drastically reduces image size.

Q: How do you handle secrets in Docker?

A: Use Docker secrets (Swarm) or environment files (.env). Never hardcode secrets in Dockerfile or image layers — they're visible in image history.

■ QUICK REVISION

- Image = blueprint; Container = running instance
- Multi-stage builds: separate builder from runtime image
- Volumes: persist data beyond container lifecycle
- docker-compose: orchestrates multi-container apps locally
- EXPOSE documents the port; -p actually maps it to the host

Chapter 23: CI/CD Pipeline

23.1 CI/CD Concepts

Stage	Description	Tools
Source Control	Code pushed to repo triggers pipeline	GitHub, GitLab, Bitbucket
Build	Compile code, resolve dependencies	Maven, Gradle, npm
Test	Unit, integration, and e2e tests	JUnit, Jest, Selenium
Code Quality	Static analysis, code coverage	SonarQube, Checkstyle
Containerize	Build Docker image and push to registry	Docker, ECR, Docker Hub
Deploy (Staging)	Deploy to test environment	Kubernetes, ECS, Heroku
Deploy (Prod)	Deploy to production (manual gate or auto)	K8s, ECS, ArgoCD

23.2 GitHub Actions – Example Pipeline

```
# .github/workflows/ci-cd.yml
name: CI/CD Pipeline
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '21', distribution: 'temurin' }
      - name: Run Tests
        run: mvn test
      - name: Upload Coverage
        uses: codecov/codecov-action@v3

  build-and-push:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4
      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKER_USERNAME }
          password: ${ secrets.DOCKER_PASSWORD }
      - name: Build & Push
        uses: docker/build-push-action@v5
        with:
          push: true
          tags: myuser/myapp:${ github.sha }

  deploy:
```

```

needs: build-and-push
runs-on: ubuntu-latest
steps:
  - name: Deploy to Production
    run: |
      kubectl set image deployment/myapp \
        myapp=myuser/myapp:${{ github.sha }}

```

23.3 Deployment Strategies

Strategy	Description	Downtime	Risk
Blue/Green	Run two identical envs; switch traffic instantly	None	Low
Rolling	Replace old pods with new ones gradually	None	Medium
Canary	Route small % of traffic to new version first	None	Very Low
Recreate	Stop old, start new	Yes	High
Feature Flags	Deploy code but toggle features on/off	None	Very Low

■ Most Asked Interview Questions

Q: What is the difference between CI and CD?

A: CI (Continuous Integration): automatically build and test code on every commit. CD can mean Continuous Delivery (auto to staging, manual to prod) or Continuous Deployment (auto all the way to prod).

Q: How do you roll back a failed deployment?

A: Blue/Green: switch traffic back to blue. Kubernetes: `kubectl rollout undo`. GitOps: revert the Git commit and the CD tool re-applies.

Q: What is a canary deployment?

A: Route a small percentage of users (e.g., 5%) to the new version. Monitor metrics. If no issues, gradually increase to 100%. Minimizes blast radius of bugs.

■ QUICK REVISION

- CI: build + test on every push. CD: auto-deploy to staging or prod.
- GitHub Actions: workflows triggered by push, PR, schedule, etc.
- Blue/Green: zero downtime, instant rollback, doubles infrastructure cost
- Canary: safest, gradual rollout with real-user monitoring
- Secrets: use repo secrets / vault — never in YAML files

SECTION 3: STUDY PLANS

15-Day DSA Roadmap

Day	Chapter	Topics Covered
Day 1	Linked Lists	Singly Linked List, Reverse, Middle of Linked List
Day 2	Linked Lists	Doubly Linked List, Detect Cycle, Merge Two Sorted Lists
Day 3	Stack & Queues	Stack Implementation, Valid Parentheses, Next Greater Element
Day 4	Stack & Queues	Queue Implementation, Sliding Window Maximum, LRU Cache
Day 5	Trees & Graphs	Binary Tree Traversals, Maximum Depth, Lowest Common Ancestor
Day 6	Trees & Graphs	Binary Search Tree, Validate BST, Graph BFS & DFS
Day 7	Trees & Graphs	Shortest Path (Dijkstra), Minimum Spanning Tree
Day 8	Backtracking	Introduction, N-Queens Problem
Day 9	Backtracking	Combinations, Permutations, Subsets
Day 10	Backtracking	Word Search, Palindrome Partitioning
Day 11	Dynamic Programming	Intro to DP, Memoization, Climbing Stairs, Fibonacci
Day 12	Dynamic Programming	0/1 Knapsack, LCS, Edit Distance
Day 13	Dynamic Programming	Matrix Chain Multiplication, Advanced DP on Trees/Graphs
Day 14	Greedy Algorithms	Activity Selection, Fractional Knapsack, Assign Cookies
Day 15	Greedy Algorithms	Minimum Platforms, Job Sequencing, Advanced Greedy

15-Day Java + Full Stack Roadmap

Day	Chapter	Topics Covered
Day 1	Java Fundamentals	JVM Architecture, JDK, Data Types, Arrays
Day 2	OOPs Deep Dive	Inheritance, Interface, Polymorphism
Day 3	Collections & Generics	List, Set, Map, Hashing Mechanics
Day 4	Stream API & Lambda	Functional Interfaces, Filters, Maps
Day 5	Relational DB & JDBC	Joins, Indexing, JDBC Connectivity
Day 6	Spring Boot Core	IoC, DI, Spring Bean Lifecycle
Day 7	Data JPA & Hibernate	Entity Mapping, JPQL, Repositories
Day 8	REST API Design	Controller, DTOs, Error Handling
Day 9	Spring Security	JWT, OAuth2, Filter Chain
Day 10	React Basics	JSX, Props, State, Component Lifecycle
Day 11	Advanced Hooks	useEffect, useContext, useMemo
Day 12	React Routing & Forms	React Router, Formik, Axios

Day	Chapter	Topics Covered
Day 13	Full Stack Integration	Connecting React to Spring Boot, CORS
Day 14	Project Build – I	Backend Logic & DB Setup
Day 15	Project Build – II	Frontend Integration & Testing

Final Interview Tips

- Always clarify the problem before coding — ask about input constraints, edge cases, and expected output.
- Think aloud. Interviewers evaluate your thought process, not just the final answer.
- Start with a brute-force solution, then optimize. State the complexity of each approach.
- Practice on a whiteboard or paper — not just in an IDE.
- Review your code for edge cases: empty input, single element, negative numbers, overflow.
- For system design: clarify scale, then components (API → DB → Cache → Queue → CDN).
- For behavioral questions: use STAR format (Situation, Task, Action, Result).
- Solve at least 2–3 LeetCode problems per day in the final month before interviews.

■ **TIP: Target companies: practice their tagged questions on LeetCode 3–4 weeks before the interview.**

■ **NOTE: Resources: NeetCode 150, Striver's A-Z DSA Sheet, System Design Primer (GitHub).**